

Experiment 6

Create Web page for E-commerce Product Page: Use of CSS for style text, colors, fonts, margin, border, padding, width, and height.

Objectives

1. To design a simple **e-commerce product page** using HTML and CSS.
2. To demonstrate the use of **text, colors, and fonts** for better presentation.
3. To apply **margin, padding, and borders** for proper spacing and structure.
4. To set **width and height** for images and containers to create a neat layout.

Tools Required

- Computer / Laptop with any OS
- **Text Editor** (VS Code, Sublime Text, Notepad++)
- **Web Browser** (Chrome, Edge, Firefox)
- Internet connection (for YouTube embedding)

Code Implementation

Here's the **HTML code**:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Product Page - Simple</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header class="site-header">
    <h1>ShopEasy</h1>
  </header>

  <main class="container">
    <section class="product-card">
      
      <h2 class="product-title">Classic Cotton T-Shirt</h2>
      <p class="product-desc">Soft, breathable cotton. Regular fit. Available in multiple colors.</p>

      <div class="price-row">
        <span class="price">₹799</span>
        <span class="old-price">₹1,199</span>
      </div>

      <button class="add-cart">Add to Cart</button>
```

```
</section>
</main>

</body>
</html>
```

style.css

```
/* simple reset */
* { box-sizing: border-box; margin: 0; padding: 0; font-family: Arial, sans-serif; }

/* layout */
body { background: #f7f8fb; color: #111; padding: 20px; }

/* header (block element) */
.site-header {
  display: block;          /* block */
  background: #222;
  color: #fff;
  padding: 12px;
  text-align: center;
  margin-bottom: 18px;    /* margin */
}

/* container */
.container { max-width: 420px; margin: 0 auto; }

/* product card */
.product-card {
  background: #fff;
  border: 1px solid #e0e4ef; /* border */
  padding: 12px;           /* padding */
  width: 100%;             /* width */
  height: auto;            /* height (auto) */
  border-radius: 6px;
}

/* image sizing */
.product-image {
  width: 100%;             /* width */
  height: 200px;          /* height */
  object-fit: cover;
  display: block;         /* block-level image */
  margin-bottom: 10px;
  border: 1px solid #ddd; /* border */
}
```

```
}

/* text styling */
.product-title {
  font-size: 18px;          /* text / font */
  margin-bottom: 6px;
}

.product-desc {
  font-size: 14px;
  color: #444;              /* color */
  margin-bottom: 10px;
}

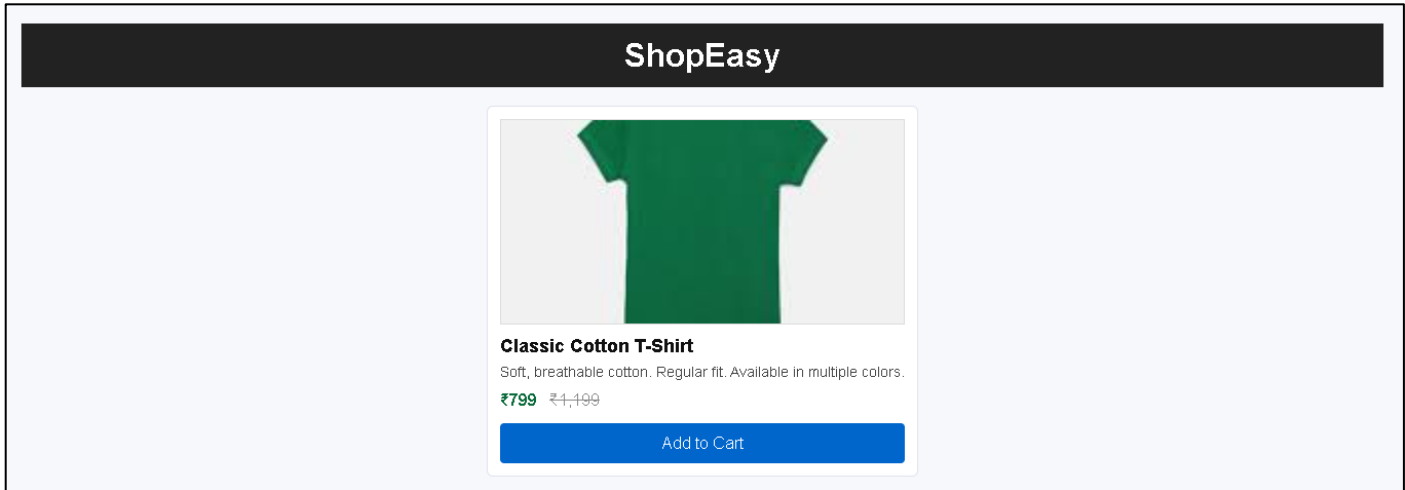
/* price row */
.price-row { margin-bottom: 12px; }
.price {
  font-weight: 700;
  color: #0b6b3a;
  margin-right: 8px;        /* margin */
  display: inline;          /* inline */
}

.old-price {
  color: #888;
  text-decoration: line-through;
  display: inline;          /* inline */
}

/* button */
.add-cart {
  display: block;           /* block button fills available width */
  width: 100%;              /* width */
  padding: 10px;            /* padding */
  background: #0066cc;      /* color */
  color: #fff;
  border: none;
  border-radius: 4px;        /* border radius (visual) */
  cursor: pointer;
  font-size: 16px;          /* font */
}

/* small responsive tweak */
@media (max-width: 20px) {
  .product-image { height: 60px; } /* height change */
}
```

Output:



Outcome

- Successfully created a **E-commerce Product Page** using only HTML & CSS
 - Applied **CSS** for style text, colors, fonts, margin, border, padding, width, and height
-

Experiment 7

Create Web page for Navigation Menu: Use of block, inline, inline-block, float, and clear

Objectives

4. To design a simple navigation menu on a webpage.
 5. To apply form attributes:
 - block for full-width sections like header
 - inline for text elements within a line.
 - inline-block for horizontal menu items that keep block features.
 6. To apply float property for arranging navigation links side by side.
 7. To use clear property to manage layout and prevent content overlap after floated elements.
-

Tools Required

- Text Editor (Notepad, VS Code, Sublime Text)
 - Web Browser (Chrome, Edge, Firefox)
-

Code Implementation

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Simple Navigation Menu</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <!-- Block Example -->
  <header>
    My <span>Website</span> Navigation
  </header>

  <!-- Inline-block Example -->
  <nav class="inline-block-menu">
    <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#">About</a></li>
      <li><a href="#">Services</a></li>
      <li><a href="#">Contact</a></li>
```

```

    </ul>
</nav>

<!-- Float + Clear Example -->
<nav class="float-menu">
    <ul>
        <li><a href="#">Dashboard</a></li>
        <li><a href="#">Profile</a></li>
        <li><a href="#">Messages</a></li>
        <li><a href="#">Logout</a></li>
    </ul>
    <div class="clear"></div>
</nav>

</body>
</html>

```

Here's the **CSS code**:

style.css

```

/* Block Example */
header {
    display: block;
    background: lightgray;
    padding: 10px;
    text-align: center;
}

/* Inline Example */
header span {
    display: inline;
    color: red;
}

/* Inline-block Navigation */
.inline-block-menu ul {
    list-style: none;
    padding: 0;
}
.inline-block-menu li {
    display: inline-block;
    margin-right: 10px;
}
.inline-block-menu a {
    text-decoration: none;

```

```
padding: 5px;
background: lightblue;
}

/* Float Navigation */
.float-menu ul {
  list-style: none;
  padding: 0;
}
.float-menu li {
  float: left;
  margin-right: 10px;
}
.float-menu a {
  text-decoration: none;
  padding: 5px;
  background: lightgreen;
}

/* Clear Example */
.clear {
  clear: both;
}
```

Output



Experiment 8:

Create Web page for Student Profile Card: design layouts using flexbox (justify-content, align-items, flex-direction).

Objective: To design layouts using flexbox (justify-content, align-items, flex-direction).

Tools Required

- Text Editor (Notepad, VS Code, Sublime Text)
 - Web Browser (Chrome, Edge, Firefox)
-

Theory:

Flexbox (**Flexible Box Layout**) is a CSS layout model that allows easy alignment and distribution of elements inside a container. It is especially useful for responsive design.

Key properties:

1. **display: flex;** → Defines a flex container.
2. **flex-direction:**
 - row → Items placed left to right (default).
 - column → Items placed top to bottom.
 - row-reverse / column-reverse → Reverse order.
3. **justify-content:** (aligns items on the main axis)
 - flex-start → Items at the beginning.
 - center → Items centered.
 - flex-end → Items at the end.
 - space-between, space-around, space-evenly → Distributes space.
4. **align-items:** (aligns items on the cross axis)
 - flex-start, center, flex-end, stretch, baseline.
5. **flex-wrap:** → Moves items to the next line if space is insufficient.

In this experiment, we apply **flexbox** to arrange multiple student profile cards neatly.

Code Implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Experiment - Flexbox Student Profile Card</title>
  <style>
    body{
      font-family: Arial, sans-serif;
      margin: 20px;
      background: #f9f9f9;
    }
  </style>
</head>
<body>
```



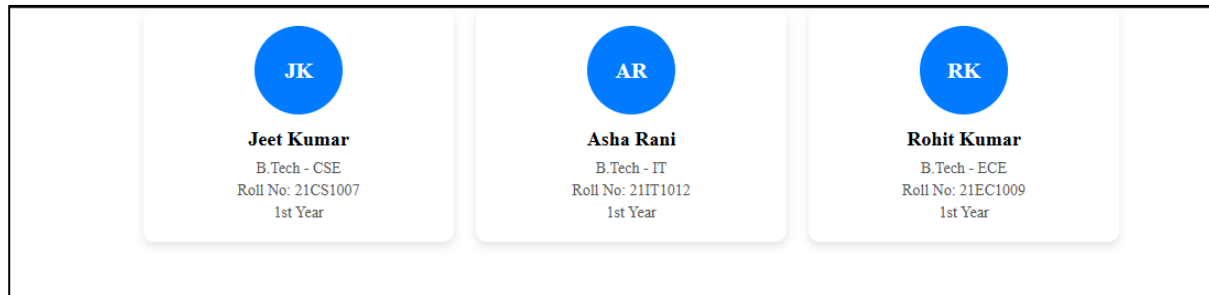
```

h2{text-align: center;}
.container{
  display: flex;           /* Flexbox container */
  flex-direction: row;     /* Arrange in a row */
  justify-content: center; /* Center horizontally */
  align-items: center;     /* Align vertically */
  gap: 20px;              /* Space between cards */
  flex-wrap: wrap;        /* Wrap to next line if needed */
}
.card{
  background: #fff;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  width: 220px;
  text-align: center;
  padding: 16px;
}
.avatar{
  width: 70px; height: 70px;
  background: #007bff; color: #fff;
  border-radius: 50%;
  display: flex; justify-content: center; align-items: center;
  font-size: 20px; margin: 0 auto 12px;
}
.name{ font-weight: bold; font-size: 18px; }
.detail{ font-size: 14px; color: #555; margin: 2px 0; }
</style>
</head>
<body>
<h2>Student Profile Cards</h2>
<div class="container">
  <div class="card">
    <div class="avatar">JK</div>
    <div class="name">Jeet Kumar</div>
    <div class="detail">B.Tech - CSE</div>
    <div class="detail">Roll No: 21CS1007</div>
    <div class="detail">1st Year</div>
  </div>
  <div class="card">
    <div class="avatar">AR</div>
    <div class="name">Asha Rani</div>
    <div class="detail">B.Tech - IT</div>
    <div class="detail">Roll No: 21IT1012</div>
    <div class="detail">1st Year</div>
  </div>
  <div class="card">
    <div class="avatar">RK</div>
    <div class="name">Rohit Kumar</div>
    <div class="detail">B.Tech - ECE</div>
    <div class="detail">Roll No: 21EC1009</div>
    <div class="detail">1st Year</div>
  </div>
</div>
</body>
</html>

```

Output:

Three student profile cards arranged in a row, aligned at the center of the page.



Outcome:

- By changing flex-direction, cards can be aligned in a **row** or **column**.
 - Using justify-content and align-items, we can change the alignment of cards across the page.
 - Successfully created a **Student Profile Card layout** using **Flexbox**.
-

Experiment 9:

Create Web page for Responsive Product Grid: create mobile-friendly designs with breakpoints.

Objective: To design a **Responsive Product Grid webpage** using **CSS Grid** and **media queries (breakpoints)** for mobile-friendly layouts.

Tools Required

- Text Editor (Notepad, VS Code, Sublime Text)
- Web Browser (Chrome, Edge, Firefox)

Theory:

A responsive web design adjusts automatically to different screen sizes (desktop, tablet, mobile).

- **CSS Grid:** Used to create structured layouts in rows and columns.
- **Breakpoints:** Specific screen width values defined in **media queries** to apply different CSS rules for different devices.

Common breakpoints:

- **Desktop (large screens):** $\geq 1024\text{px}$
- **Tablet (medium screens):** $600\text{px} - 1024\text{px}$
- **Mobile (small screens):** $\leq 600\text{px}$

In this experiment:

- Grid has **4 columns** on desktop,
- **2 columns** on tablets,
- **1 column** on mobiles.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Product Grid</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 0;
      background: #f5f5f5;
      padding: 20px;
    }

    h2 {
      text-align: center;
      margin-bottom: 20px;
    }
```

```
/* Product grid */
.grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr); /* 4 columns for large screens */
  gap: 20px;
}

/* Product card */
.card {
  background: #fff;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  padding: 16px;
  text-align: center;
}

.card img {
  width: 100%;
  height: 150px;
  object-fit: cover;
  border-radius: 6px;
  margin-bottom: 10px;
}

.card h3 {
  font-size: 18px;
  margin: 8px 0;
}

.card p {
  font-size: 14px;
  color: #555;
  margin: 4px 0;
}

.price {
  font-weight: bold;
  color: #007bff;
}

/* Responsive Breakpoints */
@media (max-width: 1024px) {
  .grid {
    grid-template-columns: repeat(2, 1fr); /* 2 columns for tablets */
  }
}

@media (max-width: 600px) {
  .grid {
    grid-template-columns: 1fr; /* 1 column for mobile */
  }
}
</style>
</head>
<body>
<h2>Responsive Product Grid</h2>
```

```

<div class="grid">
  <div class="card">
    
    <h3>Product 1</h3>
    <p>High-quality item with great features.</p>
    <p class="price">₹499</p>
  </div>

  <div class="card">
    
    <h3>Product 2</h3>
    <p>Durable and stylish for daily use.</p>
    <p class="price">₹799</p>
  </div>

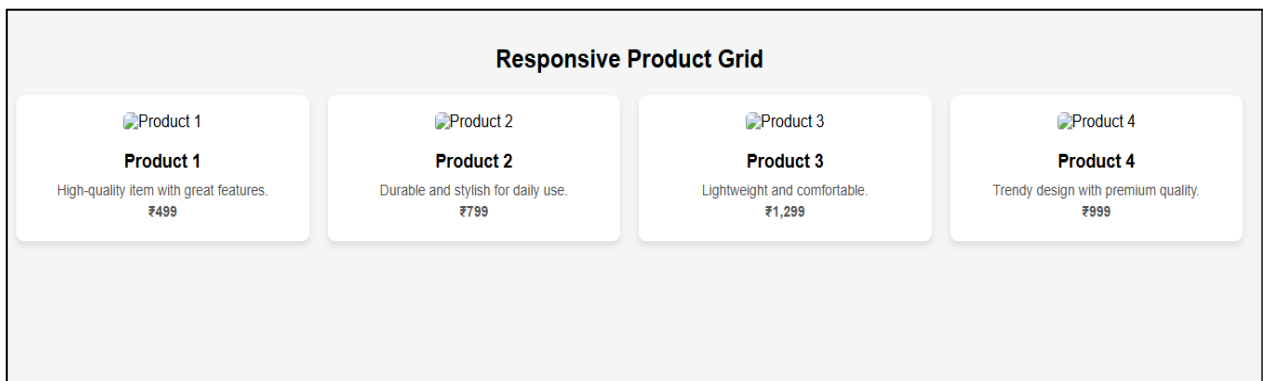
  <div class="card">
    
    <h3>Product 3</h3>
    <p>Lightweight and comfortable.</p>
    <p class="price">₹1,299</p>
  </div>

  <div class="card">
    
    <h3>Product 4</h3>
    <p>Trendy design with premium quality.</p>
    <p class="price">₹999</p>
  </div>
</div>
</body>
</html>

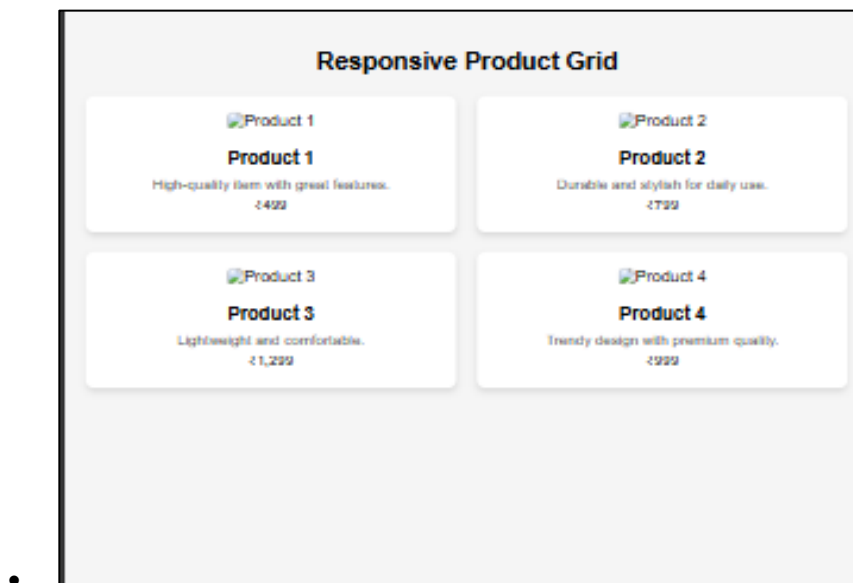
```

Output:

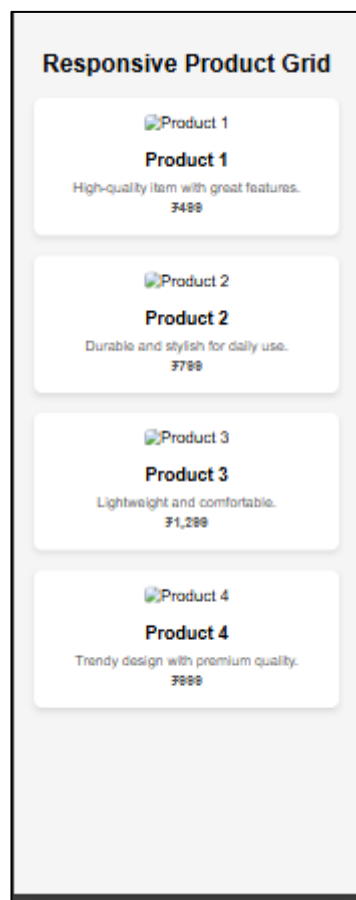
- On **desktop**: 4 product cards per row.



- On **tablet ($\leq 1024\text{px}$)**: 2 product cards per row.



- On **mobile ($\leq 600\text{px}$)**: 1 product card per row.



Outcome:

- The layout changes based on screen size. Breakpoints ensure readability and user-friendly design across devices.
 - Successfully created a **Responsive Product Grid** webpage using **CSS Grid and media queries**.
-

Experiment 10

Create a Responsive E-commerce Style Navbar using HTML, CSS, Flexbox, and Media Queries.

Objectives

1. To design a **navigation bar (navbar)** for an e-commerce website.
 2. To apply **CSS display properties** (block, inline, inline-block, float, clear).
 3. To implement **Flexbox properties** (justify-content, align-items, flex-direction).
 4. To use **relative units** (px, %, em, rem, vh, vw) for responsiveness.
 5. To apply **media queries and breakpoints** for **mobile-first responsive design**.
-

Tools Required

- Text Editor (VS Code / Sublime Text / Notepad++)
 - Web Browser (Chrome / Firefox / Edge)
-

Theory

- **Navigation Bar (Navbar):** A horizontal or vertical menu allowing users to browse site sections.
 - **Display in CSS:**
 - block → element takes full width.
 - inline → element takes only needed width.
 - inline-block → inline element but allows width/height.
 - float and clear → for positioning elements.
 - **Flexbox:**
 - display: flex; → creates a flex container.
 - justify-content → aligns items horizontally.
 - align-items → aligns items vertically.
 - flex-direction → row/column layout.
 - **Responsive Design:**
 - **Units** (% , em, rem, vh, vw) adjust with screen size.
 - **Media Queries:** Example →
 - @media (max-width: 768px) { ... }

lets us apply different styles on smaller devices.
 - **Mobile-First Design:** Start with styles for small screens, then enhance for larger screens.
-

Code Implementation (HTML + CSS)

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>E-commerce Navbar</title>
  <link rel="stylesheet" href="exp10style.css">
</head>
<body>

  <!-- Navbar -->
  <nav class="navbar">
    <div class="logo">ShopEase</div>
    <ul class="nav-links">
      <li><a href="#">Home</a></li>
      <li><a href="#">Products</a></li>
      <li><a href="#">Categories</a></li>
      <li><a href="#">Deals</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
    <div class="actions">
      <a href="#">Cart 🛒</a>
      <a href="#">Login</a>
    </div>
  </nav>

  <section>
    <h1>Welcome to ShopEase!</h1>
    <p>Your one-stop destination for all shopping needs.</p>
  </section>

</body>
</html>
```

Here's the **CSS code**:

exp10style.css

```
/* Reset */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```



```
/* Navbar Styling */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: #222;
  color: white;
  padding: 15px 20px;
}

.logo {
  font-size: 1.5rem;
  font-weight: bold;
}

.nav-links {
  display: flex;
  list-style: none;
}

.nav-links li {
  margin: 0 15px;
}

.nav-links a {
  text-decoration: none;
  color: white;
  font-size: 1rem;
}

.actions a {
  margin-left: 15px;
  color: white;
  text-decoration: none;
  font-weight: bold;
}

/* Responsive Design */
@media (max-width: 768px) {
  .nav-links {
    display: none; /* Hide menu on smaller screens */
  }

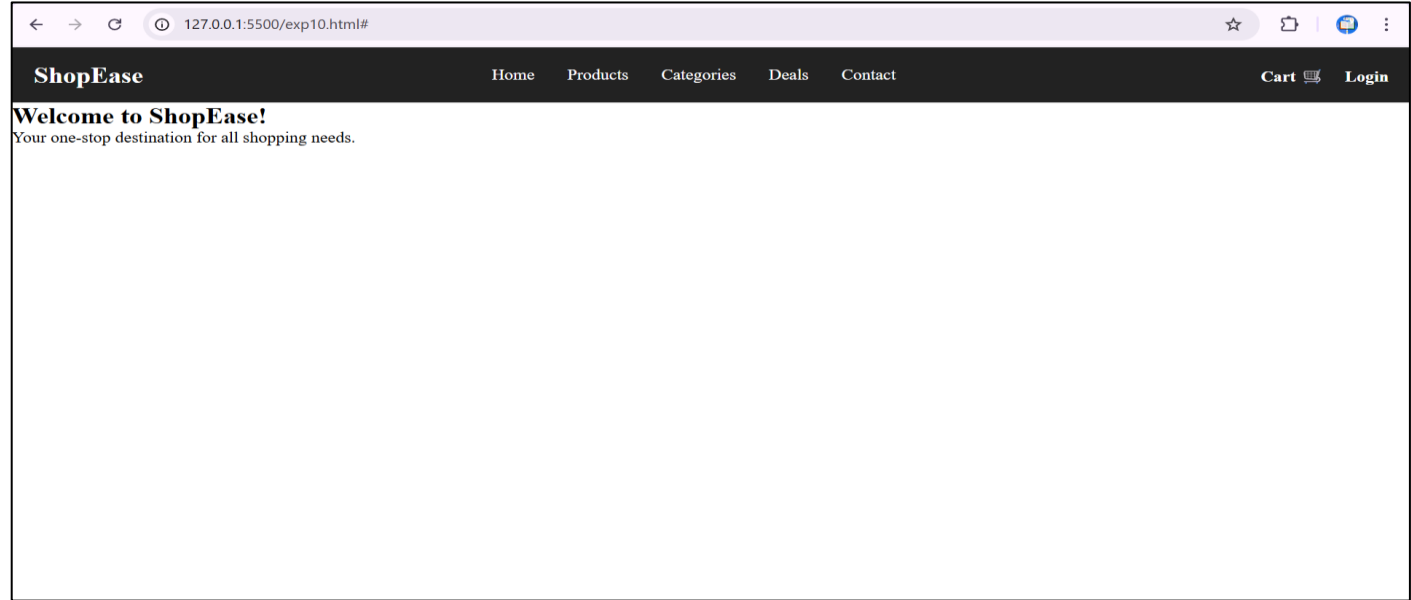
  .actions {
    font-size: 0.9rem;
  }
}
```

```
}

@media (min-width: 769px) and (max-width: 1024px) {
  .nav-links a {
    font-size: 0.9rem;
  }
}

@media (min-width: 1025px) {
  .nav-links a {
    font-size: 1rem;
  }
}
```

Output



Outcome

A responsive **E-commerce style navbar** with logo, menu items, and cart/login buttons.

Experiment: Extended – Responsive E-commerce Navbar with Hamburger Menu

Objectives

- Create a responsive **e-commerce style navbar** with a hamburger menu.
 - Learn how to toggle navigation menu on mobile screens using only **HTML + CSS**.
 - Practice **flexbox**, **media queries**, and **mobile-first design**.
-

Tools Required

- Text Editor (VS Code / Notepad++ / Sublime).
 - Web Browser (Chrome / Firefox / Edge).
-

Expected Outcome

- A responsive navbar with **logo + links + cart/login**.
 - On **desktop/tablet** → full menu visible.
 - On **mobile screen** → only logo + ☰ (hamburger). Clicking ☰ will show/hide the menu.
-

Theory

1. **Checkbox Hack**
 - A hidden `<input type="checkbox">` is used.
 - A `<label>` with ☰ icon is linked to the checkbox.
 - When checkbox is checked → CSS: checked selector shows the menu.
 2. **Flexbox Layout**
 - Navbar divided into **logo, links, and actions**.
 - `justify-content: space-between; align-items: center; .`
 3. **Media Queries**
 - Hide links in small screens.
 - Show hamburger icon instead.
 - On click, menu slides down.
-

Code Implementation

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>E-commerce Navbar Extended</title>
```

```

<link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- Navbar -->
  <nav class="navbar">
    <div class="logo">ShopEase</div>
    <!-- Hamburger menu button (checkbox hack) -->
    <input type="checkbox" id="menu-toggle">
    <label for="menu-toggle" class="hamburger">&#9776;</label>
    <!-- Navigation Links -->
    <ul class="nav-links">
      <li><a href="#">Home</a></li>
      <li><a href="#">Products</a></li>
      <li><a href="#">Categories</a></li>
      <li><a href="#">Deals</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
    <!-- Actions -->
    <div class="actions">
      <a href="#">Cart 🛒</a>
      <a href="#">Login</a>
    </div>
  </nav>

  <!-- Sample Content -->
  <section>
    <h1>Welcome to ShopEase!</h1>
    <p>Your one-stop destination for all shopping needs.</p>
  </section>
</body>
</html>

```

style.css

```

/* Reset */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* Navbar Container */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;

```

```
background-color: #222;  
color: white;  
padding: 15px 20px;  
position: relative;  
}
```

```
/* Logo */
```

```
.logo {  
  font-size: 1.5rem;  
  font-weight: bold;  
}
```

```
/* Nav Links */
```

```
.nav-links {  
  display: flex;  
  list-style: none;  
}
```

```
.nav-links li {  
  margin: 0 15px;  
}
```

```
.nav-links a {  
  text-decoration: none;  
  color: white;  
  font-size: 1rem;  
}
```

```
/* Actions */
```

```
.actions a {  
  margin-left: 15px;  
  color: white;  
  text-decoration: none;  
  font-weight: bold;  
}
```

```
/* Hamburger Menu */
```

```
.hamburger {  
  display: none;  
  font-size: 2rem;  
  cursor: pointer;  
}
```

```
/* Hide checkbox */
```

```
#menu-toggle {  
  display: none;
```

```

}

/* 📱 Mobile-first Design */
@media (max-width: 768px) {
  .nav-links {
    display: none;
    position: absolute;
    top: 60px;
    left: 0;
    right: 0;
    flex-direction: column;
    background-color: #333;
    padding: 15px 0;
  }

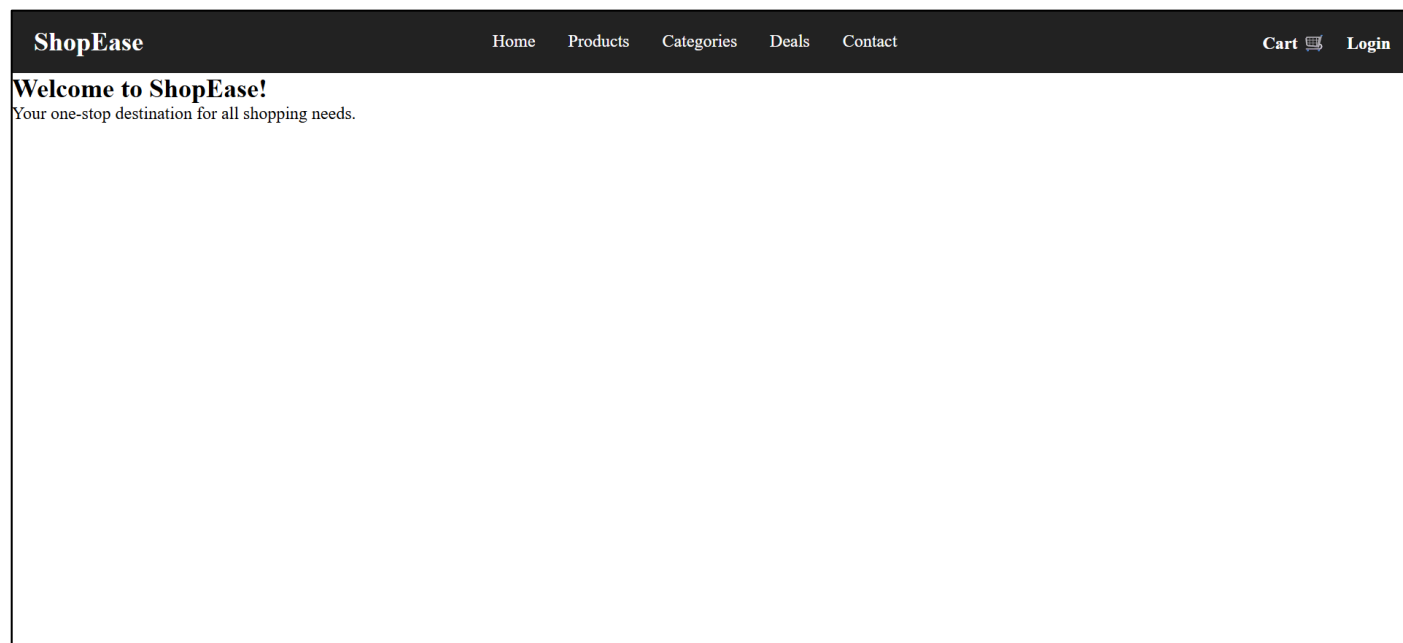
  .nav-links li {
    margin: 10px 0;
    text-align: center;
  }

  .hamburger {
    display: block;
  }

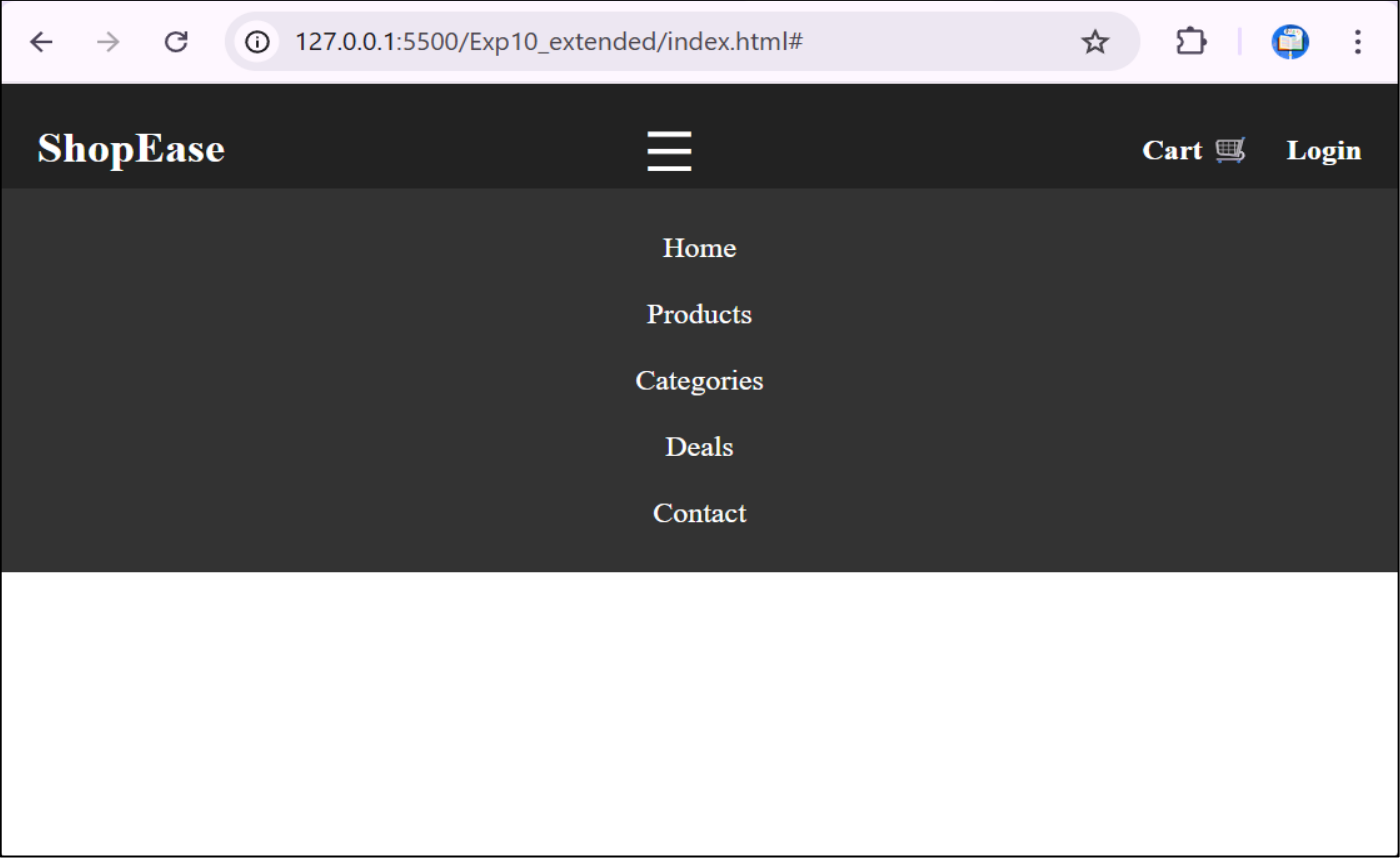
  /* Show menu when checkbox is checked */
  #menu-toggle:checked + .hamburger + .nav-links {
    display: flex;
  }
}

```

Output:



Mobile View (Collapsed)



Outcome

- Successfully created a **responsive e-commerce style navbar**.
 - Used **display properties, flexbox, relative units, and media queries**.
 - Implemented a **mobile-first design** with hamburger menu.
 - Learned how to switch between **desktop and mobile layouts** using CSS.
-